Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
int i= 1; int s =1; while(s <= n)
{
i++;
s += i;
}
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:
A positive Integer n
Output:
Print the value of the counter variable
CODE:

```
#include <stdio.h>

int main()
{
int n;

int i = 1;
int s = 1;
int c = 3;
scanf("%d",&n);

while (s <= n)
{
c++;
i++;
c++;
s+=i;
c++;
}
```

```
printf("%d", c);

return 0;
}
```

2. Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
if(n==1)
{
printf("*");
}
else
{
for(int i=1; i<=n; i++)
{
for(int j=1; j<=n; j++)
{
printf("*");
printf("*");
break;
}
}
}
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

CODE:

```
#include <stdio.h>

int main()
{
int n;
```

```
scanf("%d", &n);

int count = 1;

if (n == 1)
{
count++;
}
else
{
count++;

for (int i = 1; i <= n; i++)
{
count++;
count++;

for (int j = 1; j <= n; j++)
{
count++;

count++;

count++;
break;
}
}
}

printf("%d", count);

return 0;
}
```

3. Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
for (i = 1; i <= num;++i)
```

```
{
if (num % i== 0)
{
printf("%d ", i);
}
}
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:
A positive Integer n
Output:
Print the value of the counter variable
CODE:

```
#include <stdio.h>

int main()
{
int num,i,counter=0;

scanf("%d",&num);

for ( i = 1; i <= num; i++)
{
counter++;

if (num % i == 0)
{
counter++;
}
}
counter +=num+1;

printf("%d", counter);
```

```
return 0;
}
```

4. Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
int c= 0;
for(int i=n/2; i<n; i++)
for(int j=1; j<n; j = 2 * j)
for(int k=1; k<n; k = k * 2)
c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:
A positive Integer n
Output:
Print the value of the counter variable
CODE:

```
#include <stdio.h>

int main()
{
int n;
scanf("%d", &n);

int c = 0;
int count = 0;

count++; // i = n/2

for (int i = n / 2; i < n; i++)
{
count++; // outer loop condition

for (int j = 1; j < n; j = 2 * j)
{
```

```
count++; // middle loop condition

for (int k = 1; k < n; k = k * 2)
{
count++;
c++;
count++;
}

count++;
}

count++;
}

count++;

printf("%d", count);

return 0;
}
```

5.Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
int rev = 0, remainder;
while (n != 0)
{
remainder = n % 10;
rev = rev * 10 + remainder;
n/= 10;

}
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf()

statements.

Input:
A positive Integer n
Output:
Print the value of the counter variable
CODE:

```
#include<stdio.h>
int main()
{
int n,rev=0,remainder,counter=3;
scanf("%d",&n);
while(n!=0)
{
counter+=4;
remainder=n%10;
rev=rev*10+remainder;
n=n/10;
}
printf("%d",counter);
return 0;
}
```